Sonnette d'interphone SIP

Rapport de projet — Asterisk, baresip et une interface en Rust
Projet linux-embarque · 19 octobre 2026

En une phrase. Une vraie sonnette d'immeuble : un vrai central téléphonique (Asterisk), de vrais postes SIP (baresip), un menu vocal et de vrais messages enregistrés, pilotés depuis un tableau de bord en terminal écrit en Rust. **Aucun mode simulé** : les appels, les sonneries, les messages et les fichiers audio sont réels.

1. Objectif

Reproduire une sonnette d'interphone complète et vérifiable. Le visiteur se présente à l'entrée, choisit un résident dans un menu vocal, puis soit lui parle, soit lui laisse un message enregistré. Le résident dispose d'un tableau de bord qui affiche ses messages et permet de les réécouter. Le projet tient dans un dépôt unique, lançable sur n'importe quel Linux avec Docker.

2. Le déroulement, de bout en bout

Ce que fait la sonnette, vu de l'utilisateur :

- **Le visiteur compose 9999** et entend le menu vocal.
- **Il choisit un résident** : 1 Hadji, 2 Johnson, 3 Miora, 4 Ulrich.
- **Deuxième niveau** : **1** pour appeler maintenant, **2** pour laisser un message.
- **« Appeler maintenant »** fait sonner le poste choisi **4 fois (24 s)** ; si personne ne répond, le visiteur est invité à laisser un message.
- **« Laisser un message »** : invite, bip, enregistrement. Le visiteur parle puis raccroche — le message est conservé.
- **Côté résident**, le message apparaît tout seul dans son panneau ; la touche **P** le réécoute, **D** le supprime.
- **Appeler un poste directement** (1003 par exemple) revient au même : 4 sonneries, puis un message si personne ne décroche. Un appel **refusé** se termine simplement.

3. Architecture

```
1. Le visiteur compose 9999 sur son poste SIP (baresip « hadji »)
      |
      v
2. Asterisk (conteneur « doorbell-device »)
     - joue le menu vocal et attend les touches (DTMF)
     - fait sonner le poste choisi (baresip « miora »), 4 coups
     - enregistre la voix du visiteur
      |
      v
3. Fichier WAV écrit dans
   /var/spool/asterisk/monitor/1003-<horodatage>-<id>.wav
      |
      v
4. L'interface TUI (Rust) liste ce fichier dans le panneau « Miora 1003 »
     -> le résident appuie sur P pour le réécouter
```

4. Les constituants

Constituant	Ce dont il s'occupe
Asterisk 20.9.3 (conteneur Docker)	Le central : enregistrement des postes, menu vocal, sonnerie, enregistrement des appels et des messages, messagerie vocale.
baresip 1.1.0 (4 instances)	Les postes téléphoniques : appels SIP, RTP, DTMF, sonnerie locale et périphériques audio. Une instance par résident, chacune sur son propre port.
Interface TUI (Rust, ratatui)	Cinq panneaux (un par résident + CONFIG) : composition des numéros, liste des messages en direct, lecture, suppression, direction audio de chaque appel.
PipeWire / ALSA	Le micro et le haut-parleur, partagés par les quatre postes : annulation d'écho pour éviter l'effet Larsen, réglage du niveau du micro.
Scripts (Python, Bash)	Synthèse des invites vocales, test sans téléphone, lecture des enregistrements, nettoyage, démarrage.

5. La configuration

Tout ce qu'il a fallu configurer, et où. Les fichiers de *configs/* sont montés dans le conteneur Asterisk, ceux de *tui/* sont lus par l'interface et les postes.

Fichier	Ce qui y est configuré
configs/pjsip.conf	Les 4 postes : extensions 1001–1004, un port SIP par poste, codecs ulaw et alaw, un seul enregistrement actif à la fois.
configs/extensions.conf	Le plan de numérotation : menu à deux niveaux sur 9999, sonnerie de 4 coups (24 s) avant la messagerie, enregistrement du message conservé même si l'appelant raccroche, fin d'appel si l'appelé refuse, messagerie vocale sur *97.
configs/voicemail.conf	Les 4 boîtes vocales avec leurs codes confidentiels.
configs/rtp.conf	La plage RTP utilisée par Asterisk (10000–10100).
tui/conf/<poste>/config (baresip)	Un dossier par poste : port SIP et plage RTP propres, périphériques audio, fichier de sonnerie locale, sonnerie de retour désactivée.
tui/asound.conf	Le périphérique audio « doorbell » : le micro et le haut-parleur passent par l'annulation d'écho.
scripts/make-prompts.sh	La génération des invites vocales (voix neuronale) et leur conversion dans les trois formats utilisés par Asterisk (wav, ulaw, alaw).
docker/docker-compose.yml	Le conteneur Asterisk, en réseau de l'hôte pour que le RTP circule correctement.

6. Les quatre postes

Poste	Extension	Port SIP	Ports RTP	Rôle
hadji	1001	5070	20000–20014	résident
johnson	1002	5080	20015–20029	résident
miora	1003	5090	20030–20044	résident
ulrich	1004	5095	20045–20059	résident

7. Les points de configuration déterminants

- **Un port SIP par poste** : les quatre postes vivent sur la même machine, chacun écoute donc sur son propre port (5070, 5080, 5090, 5095).
- **Un seul enregistrement par poste** (*max_contacts=1*) : deux instances ne peuvent pas se disputer la même extension.
- **Le menu est audible par l'appelant** : les appels sont établis en duplex, sinon Asterisk joue les invites dans le vide.
- **Une seule voix circule entre deux personnes** : l'appelé répond en réception seule et son micro est coupé, seul l'appelant parle.
- **La sonnerie ne s'entend que chez l'appelé** : la sonnerie de retour est désactivée côté appelant.
- **Les messages sont conservés au raccrochage** : l'enregistrement garde le fichier même si l'appelant raccroche au milieu de sa phrase.
- **Un fichier par appel** : l'enregistrement de la sonnerie est supprimé si l'appel ne devient pas une conversation.
- **Invites générées hors ligne** : les fichiers audio sont livrés avec le projet, la démonstration n'a besoin d'aucun accès réseau.

8. Ce qui a été démontré

- **Le menu vocal** : composer 9999, écouter le menu, choisir une personne, puis appeler ou laisser un message.
- **L'appel** : la sonnerie dure 4 coups ; si personne ne répond, le visiteur est invité à laisser un message.
- **Le message** : invite, bip, enregistrement ; il est conservé même si le visiteur raccroche aussitôt après avoir parlé.
- **La consultation** : le message apparaît de lui-même dans le panneau du destinataire ; **P** le réécoute, **D** le supprime.
- **Le refus** : raccrocher pendant la sonnerie termine l'appel, sans messagerie et sans fichier résiduel.
- **L'appel direct** : composer 1003 joint Miora sans passer par le menu.

9. Résultats mesurés

Ce qui a été mesuré	Résultat
Durée de la sonnerie avant la messagerie	24,03 s — exactement 4 sonneries
Contenu du message enregistré	la voix restituée à l'identique (corrélation 1,000)
Niveaux audio des invites	moyenne ≈ −21 dBFS, crête ≈ −3 dBFS, aucune saturation
Fichiers produits par appel	exactement 1 : l'appel enregistré ou le message
Ce que l'appelant entend	4 sonneries, puis « leave your message after the beep »
Messages consultables dans l'interface	liste mise à jour automatiquement, réécoute immédiate

10. Comment lancer le projet

```
./demo.sh                       # menu : démarrer / arrêter / réinitialiser
cd tui && cargo run --release   # l'interface (à lancer depuis tui/)

# Faire le parcours complet sans téléphone : 9999 -> 3 (Miora) -> 2 (message)
./scripts/sip-test.py call --user hadji --target 9999 \
    --dtmf 32 --dtmf-delay 12 --audio msg.wav --audio-delay 6 --duration 34

./scripts/play-recording.sh --list    # écouter les enregistrements et les messages
```

11. Contenu du dépôt

```
linux-embarque/
├── configs/    configuration d'Asterisk (pjsip, extensions, voicemail, rtp)
├── docker/     le conteneur Asterisk et son réseau
├── sounds/     les invites vocales (custom/) et les sons d'Asterisk (en/)
├── scripts/    synthèse vocale, testeur SIP, lecture, nettoyage, démarrage
├── tui/        l'interface Rust (src/, conf/<poste>/, asound.conf)
├── docs/       guides de démonstration et ce rapport
└── tests/      plan de tests
```